
Initial State Distribution Manipulation for Off-Policy Reinforcement Learning

Veronika Nechaieva¹

Abstract

Curriculum learning over initial states has been shown to accelerate training in sparse-reward environments, but existing work evaluates these methods almost exclusively with on-policy algorithms. We test whether the same approach benefits off-policy learning, where a replay buffer already provides sample reuse.

We compare four start-state selection strategies with Double DQN on three MiniGrid grid-world environments of varying structure: a distance-based reverse curriculum, TD-error-based selection, uniform random placement, and a fixed start. We show that the reverse curriculum reaches peak performance within 10-40k steps across all environments, far ahead of the other selectors.

1. Introduction

Sparse rewards make reinforcement learning hard: the agent must explore extensively before encountering any learning signal, and without that signal it has no basis to improve its policy. A natural way to mitigate it is to start episodes closer to the goal, where success is easier, and gradually expand the starting region as the agent improves. This idea has been explored as reverse curriculum (Florensa et al., 2017) and TD-error-based start-state selection (Florensa et al., 2018), and is related to the archive-and-return strategy of Go-Explore (Ecoffet et al., 2021) and the contrastive initial state buffer of Messikommer et al. (Messikommer et al., 2024).

These methods have been evaluated primarily with on-policy algorithms (TRPO, PPO), where each trajectory is used once and the start-state distribution directly determines what the agent trains on. Off-policy methods such

as DQN already decouple data collection from training through a replay buffer. It is not obvious that controlling the start-state distribution provides additional benefit when past experience is already being reused.

We investigate this question by comparing four start-state selectors with Double DQN on three MiniGrid environments. We find that reverse curriculum provides efficiency gains even in the off-policy setting, and that TD-error-based selection suffers from additional problems but shows promise.

2. Background

Deep Q-Networks. DQN (Mnih et al., 2015) extends Q-learning into high-dimensional state spaces by approximating the action-value function $Q(s, a)$ with a neural network. It uses two mechanisms to stabilise the training: a *replay buffer* that stores past transitions and samples minibatches uniformly, decorrelating gradient updates; and a *target network*, a delayed copy of the Q-network used to compute bootstrap targets. Double DQN (Van Hasselt et al., 2016), used in this work, further combats overestimation bias by decoupling action selection from action evaluation (online vs target network).

Curriculum over initial states. In sparse-reward environments, the agent may rarely encounter the goal which makes the initial learning curve prohibitively slow. One remedy is to control *where* episodes begin. Florensa et al. (Florensa et al., 2017) propose a *reverse curriculum*: episodes initially start near the goal, and the start distribution expands outward as the agent demonstrates competence, gradually covering the full state space. Their evaluation uses TRPO, an on-policy method. Florensa et al. (Florensa et al., 2018) additionally introduce automatic goal generation and select start states where the agent’s *TD error* is highest. This directs training toward regions of maximum prediction uncertainty.

Related approaches. Go-Explore (Ecoffet et al., 2021) addresses the same hard-exploration problem by using an archive of visited states, *returning* to promising ones, and exploring from there. The Contrastive Initial State

¹University of Warsaw, Faculty of Mathematics, Informatics, and Mechanics. Correspondence to: Veronika Nechaieva <veronika.netcha@gmail.com>.

Buffer (Messikommer et al., 2024) selects replay-buffer states as episode initialisations for robotic control tasks. It uses on-policy PPO as its base learner.

Gap. A common thread across these works is the reliance on *on-policy* algorithms. For on-policy methods, each trajectory is used once and discarded, so controlling where episodes start directly determines the training distribution. For *off-policy* methods such as DQN, a replay buffer already decouples the data collection distribution from the training distribution, providing sample reuse independently of the start-state policy. It is therefore unclear whether manipulating the initial state distribution provides comparable benefit in the off-policy setting, or whether the replay buffer already captures most of the gain.

3. Method

3.1. Agent

We use a Double DQN (Van Hasselt et al., 2016) with a two-hidden-layer MLP (128 units each, ReLU activations) mapping the 3-dimensional observation to Q-values over the action space. We use Huber loss instead of MSE to reduce sensitivity to large TD errors, and clip gradients at $\|\cdot\| \leq 10$. The target network is updated via Polyak averaging ($\tau = 0.005$) after every gradient step instead of the periodic hard copy used in the original DQN. Exploration uses ϵ -greedy with linear annealing from 1.0 to 0.1 over 80% of training. The replay buffer stores 10^5 transitions. One gradient step is taken every 4 environment steps, starting after 1000 transitions. Table 1 lists the full hyperparameter set.

Table 1. Agent hyperparameters.

Parameter	Value
Hidden layers	2×128 (ReLU)
Learning rate	5×10^{-4} (Adam)
Discount γ	0.99
Batch size	128
Replay capacity	10^5
Target update τ	0.005
ϵ schedule	1.0 $\rightarrow$ 0.1 (linear, 80% of steps)
Gradient clip	max norm 10
Train every	4 steps
Learning starts	1000 steps

The choice of Double DQN over vanilla DQN is discussed in Section 4.1.

3.2. Start-State Selectors

Each episode, a selector picks a free cell as the agent’s start position. The selectors differ only in how they choose that cell; the agent and environment are otherwise identical across conditions.

Fixed. Always starts the agent at cell (1, 1) (top-left corner). This is the worst-case baseline: a single, distant start with no curriculum.

Uniform. Samples a free cell uniformly at random. This is the standard baseline: the agent sees a representative spread of start positions, but with no bias toward easier or harder ones.

Reverse. Inspired by Florensa et al. (Florensa et al., 2017). Cells are grouped into bands by BFS shortest-path distance from the goal. Training begins with starts sampled only from the closest bands. When the agent’s success rate (rolling window of 20 episodes) on the current outermost band exceeds 70%, the band expands by one distance level. To prevent short near-goal episodes from inflating the success count, only the episodes that actually start on the frontier band count toward expansion.

TDError. Inspired by Florensa et al. (Florensa et al., 2018). Maintains a per-cell exponential moving average ($\alpha = 0.1$) of absolute TD errors from training batches. Cells with higher error are sampled proportionally, with a bias toward the regions where the Q-network’s predictions are least accurate. A small floor ($\epsilon = 0.01$) prevents unvisited cells from being permanently ignored.

3.3. Environments

All environments use the MiniGrid framework (Chevalier-Boisvert et al., 2023) with a minimal observation vector (col, row, dir). The goal position is pinned (fixed across episodes) to ensure that curriculum selectors compare start states against a stable target; without pinning, MiniGrid randomises the goal on each reset, which confounds the curriculum signal. Figure 1 shows the layouts.

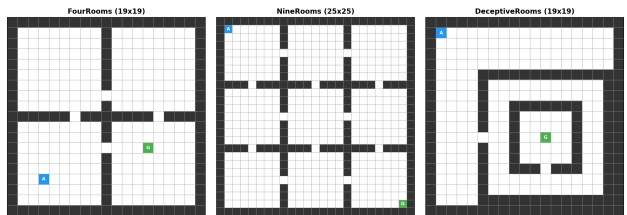


Figure 1. Grid layouts for the three environments. A (blue): default agent start; G (green): goal. Black cells are walls.

FourRooms (19×19 , standard MiniGrid). Four rooms connected by single-cell doorways. BFS shortest-path distance correlated with difficulty, as cells farther from the goal are genuinely harder to solve from.

NineRooms (25×25 , custom). A 3×3 grid of rooms with single-cell doorways between all adjacent pairs. The larger state space and additional bottlenecks increase the difficulty relative to FourRooms.

DeceptiveRooms (19×19 , custom). An open grid with a two-ring spiral surrounding the goal. Cells inside the spiral have low BFS distance but require navigating narrow corridors with multiple turns, making them harder than open-area cells at equal or greater BFS distance. This tests whether TD-error-based selection can outperform distance-based methods when BFS distance is a poor difficulty proxy.

4. Experiments

4.1. DQN Stability

Initial experiments with vanilla DQN on FourRooms showed that all four selectors would begin learning, then collapse to 0% success rate around 30-40k steps and never recover. The cause was Q-value overestimation. Double DQN (Van Hasselt et al., 2016) addresses this by using the online network to select actions and the target network to evaluate them. This approach is well-established: Van Hasselt et al. demonstrated elimination of overestimation on Atari benchmarks, and Double DQN has since become the standard. In our setting, it eliminated the collapse entirely.

Three further changes were needed: reducing the update frequency from every step to every 4 steps to prevent overfitting to the small number of transitions early in training; Huber loss with gradient clipping to reduce the impact of outlier TD errors; and replacing the hard target copy with Polyak averaging ($\tau = 0.005$) to remove discontinuity that caused oscillations in the learning curve.

4.2. Results

All tables report the peak of the rolling-50-episode success rate up to the given step count (rolling max, not instantaneous value).

FourRooms (100k steps). Reverse reaches 100% within 10k steps, dips briefly around 20-30k as the band expands through the doorway bottlenecks, then recovers and holds above 90%. TDError and Uniform track together around 30-40%, with TDError pulling slightly ahead late. Fixed stays near zero until 60k steps, then climbs slowly.

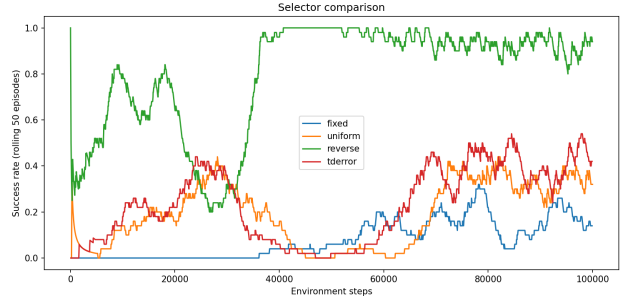


Figure 2. Learning curves on FourRooms (100k steps).

Table 2. Peak rolling-50 success rate on FourRooms.

Selector	50k	100k
Fixed	0.06	0.32
Uniform	0.44	0.44
TDError	0.44	0.54
Reverse	1.00	1.00

NineRooms (200k steps). Reverse again reaches 100% early and fluctuates between 80-100% as the band expands across the larger grid. TDError starts alongside Uniform but overtakes it after 100k steps, reaching 100% by 175k. Fixed remains at zero until around 80k, then climbs steeply to 100% by 200k. Given enough steps, all selectors except Uniform converge to full success.

Table 3. Peak rolling-50 success rate on NineRooms.

Selector	50k	100k	150k	200k
Fixed	0.00	0.16	0.42	1.00
Uniform	0.50	0.80	0.84	0.96
TDError	0.44	0.74	0.94	1.00
Reverse	1.00	1.00	1.00	1.00

DeceptiveRooms (150k steps). Reverse reaches 100% by 10k steps but shows more late-game variance than on the other environments, dipping to around 70% at times. TDError and Uniform track together until 50k, then both climb to near 100% by 90k. TDError slightly leads Uniform during the climb. Fixed never exceeds 15%. This is the only environment where TDError visibly outpaces Uniform.

The results above use the stabilised Double DQN agent (Section 4.1). Before the stability fixes, the gap between Reverse and the other selectors was even wider: Fixed never reached any reasonable performance, and Uniform showed much larger oscillations before collapsing entirely. The stabilisation narrowed the gap by making all selectors viable, but did not change the ranking.

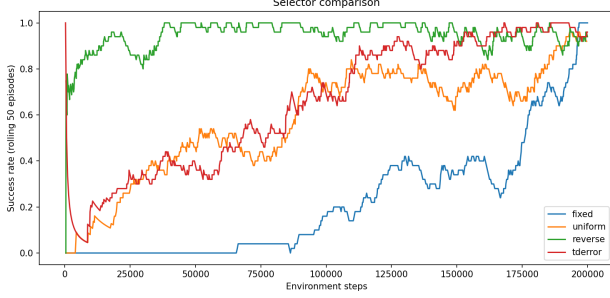


Figure 3. Learning curves on NineRooms (200k steps).

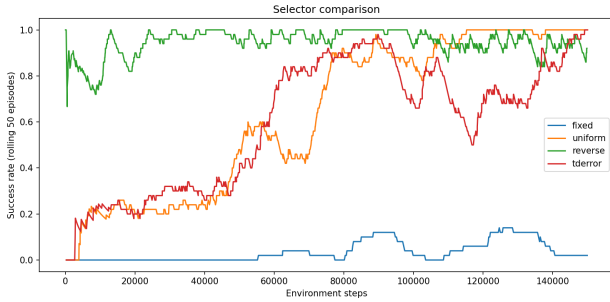


Figure 4. Learning curves on DeceptiveRooms (150k steps).

5. Discussion

Reverse curriculum consistently learns fastest - it reaches peak success rate within 10,000 to 40,000 environment steps across all three environments. There are two reasons for this advantage. First, knowledge of BFS distances and goal location is implicitly built-in in its training regimen. Second, short, near-goal episodes under it yield higher reward signal, which particularly matters during the early training stages. After enough steps, this early advantage is mostly neutralised - in NineRooms setup, all starting location selection methods ultimately reach the same performance, and likely similar picture would arise in the two other environments with more training time.

The downside of Reverse curriculum is its lower stability after reaching the stably high performance, in comparison to TDError and Uniform which nearly flatten out after reaching the 100% performance once. This behaviour does not improve with lower learning rate and/or annealing, and is likely connected to the non-stationary training distribution. The Q-network was trained predominantly on near-goal transitions, and when it gets start states from regions it has less experience with, it degrades. The replay buffer softens this effect but doesn't eliminate it, because near-goal episodes are short (few transitions per episode) while far-start episodes are long (many transitions), so the buffer gradually skews toward the newer, harder transitions.

Table 4. Peak rolling-50 success rate on DeceptiveRooms.

Selector	50k	100k	150k
Fixed	0.00	0.12	0.14
Uniform	0.48	0.98	1.00
TDError	0.38	0.98	1.00
Reverse	1.00	1.00	1.00

TD-error-based selection, despite being a more sophisticated mechanism, tracks next to Uniform in sample efficiency rather than next to Reverse. The presumed reason for this is the cold-start. TDError's per-cell error estimates begin uninformative (initialised at a small floor value), so early-game sampling is effectively uniform random. The curriculum only becomes meaningful after the agent has trained on enough transitions to produce spatially varied TD errors, by which point the Reverse selector has already built a near-complete policy. On DeceptiveRooms, however, where BFS distance is a poor proxy for navigational difficulty, TDError narrows the gap with Reverse and overtakes Uniform, which suggests that learned difficulty signals may have value when geometric priors are misleading. However, it suffers from performance instability even more so than Reverse.

The off-policy replay buffer and the start-state curriculum complement each other: replay addresses sample *reuse*, while the curriculum addresses reward *encounter* so that the agent sees successful trajectories early enough to bootstrap learning.

The main limitations of this study are the small grid environments with a simplified 3-dimensional observation, and no direct comparison to on-policy methods such as the PPO agent used in the original work (Florensa et al., 2017).

6. Conclusion

Start-state curriculum provides clear sample-efficiency gains for off-policy DQN on sparse-reward grid navigation, with the reverse curriculum reaching peak performance an order of magnitude faster than baselines across all tested environments. TD-error-based selection is less effective, but narrows the gap when distance-based priors are misleading. The replay buffer and the curriculum address different aspects of the exploration problem: the buffer provides sample reuse, the curriculum ensures reward encounter. Future work could compare directly against on-policy baselines, scale to larger or continuous environments, and explore hybrid curricula that combine distance priors with learned difficulty signals.

References

- Chevalier-Boisvert, M., Dai, B., Towers, M., de Lazcano, R., Willems, L., Lahlou, S., Pal, S., Castro, P. S., and Terry, J. Minigrid & miniworld: Modular & customizable reinforcement learning environments for goal-oriented tasks. *CoRR*, abs/2306.13831, 2023.
- Ecoffet, A., Huizinga, J., Lehman, J., Stanley, K. O., and Clune, J. First return, then explore. *Nature*, 590(7847): 580–586, 2021.
- Florensa, C., Held, D., Welling, M., and Abbeel, P. Reverse curriculum generation for reinforcement learning. *arXiv preprint arXiv:1707.05300*, 2017.
- Florensa, C., Held, D., Geng, X., and Abbeel, P. Automatic goal generation for reinforcement learning agents. *arXiv preprint arXiv:1811.11298*, 2018.
- Messikommer, N., Song, Y., and Scaramuzza, D. Contrastive initial state buffer for reinforcement learning. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2024.
- Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., et al. Human-level control through deep reinforcement learning. *Nature*, 518 (7540):529–533, 2015.
- Van Hasselt, H., Guez, A., and Silver, D. Deep reinforcement learning with double q-learning. *Proceedings of the AAAI Conference on Artificial Intelligence*, 2016.